Assignment 9.1	
Implementing Trees	
Course Code: CPE010	**Program:** Computer Engineering
Course Title: Data Structures and Algorithms	**Date Performed:** Sep 30, 2025
Section: CPE21S4	**Date Submitted:** Sep 30, 2025
Name(s): Punay, Heidee S.	**Instructor:** Engr. Jimlord Quejado

Answer the following questions:

1. Define what is a binary tree?
Binary Tree is a hierarchical data structure that has a root node where it all begins. Next to the root node are the parent nodes that have only two children, the left and the right children.

2. What are the key components of a Binary tree?
It has a root node, parent node and its children, leaf node, the edges that connect the node to other nodes and the path.

3. What are the operations that can be performed in a Binary tree?
The different operations that can be performed in a binary tree are traversal, insertion, and deletion. In traversal operation there are pre-order, in-order, and post-order traversal. First is traversal where it visits every node to process its data. Pre-order traversal visits the root node first and to the left subtree, and visits the right subtree the last. In-order traversal is opposite to pre-order, it first visits the left subtree then to the root node and visits the right subtree the last one to visit. Post-order traversal visits the root node last.

4. What are the conditions for a Binary Search Tree?
Each node in the tree only has two children which are the left and the right child. All nodes in the left subtree contain key values that are less than or equal to the key value of the parent node and vice versa.

5. Give simple sample program in C++ that uses the above algorithm. Explain how the program works.

```cpp
#include <iostream>
using namespace std;

// Define the structure of a tree node
struct Node {
    int data;
    Node* left;
    Node* right;

    // Constructor
    Node(int value) {
        data = value;
        left = right = nullptr;
    }
};

// Function to insert a node in the binary tree (BST-style)
Node* insert(Node* root, int value) {
    if (root == nullptr) {
        return new Node(value);
    }

    if (value < root->data) {
        root->left = insert(root->left, value);
```

```
24          root->left = insert(root->left, value);
25      } else {
26          root->right = insert(root->right, value);
27      }
28
29      return root;
30  }
31
32  // Inorder traversal: Left, Root, Right
33  void inorderTraversal(Node* root) {
34      if (root != nullptr) {
35          inorderTraversal(root->left);
36          cout << root->data << " ";
37          inorderTraversal(root->right);
38      }
39  }
40
41  int main() {
42      Node* root = nullptr;
43
44      // Insert values into the binary tree
45      root = insert(root, 50);
46      insert(root, 30);
```

```
46      insert(root, 30);
47      insert(root, 70);
48      insert(root, 20);
49      insert(root, 40);
50      insert(root, 60);
51      insert(root, 80);
52
53      // Perform inorder traversal
54      cout << "Inorder traversal of the binary tree: ";
55      inorderTraversal(root);
56      cout << endl;
57
58      return 0;
59  }
60
```

Output

```
Inorder traversal of the binary tree: 20 30 40 50 60 70 80


=== Code Execution Successful ===
```

Each node contains data where the values are stored. Left and right pointers to child nodes. Then adds a value to the binary tree where if the value is less than the current node it goes left and if it is greater, it goes right. It first defines each node in the tree, the value, pointer to left child and a pointer to right child. If the tree is empty root == nullptr, it creates a new node. If the value is less than the current nodes data, it inserts into the left subtree. If the value is greater than or equal, it inserts into the right subtree.